

|  | AVALIAÇÃO                                                                        |                 |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------|
|                                                                                   | <b>DISCIPLINA:</b> Fundamentos de Computação Concorrente, Paralela e Distribuída | <b>PERÍODO:</b> |
|                                                                                   | <b>PROFESSOR(A):</b> Jorge Soares                                                | <b>UNIDADE:</b> |
|                                                                                   | <b>Estudante:</b>                                                                |                 |

**Atenção:**

- Vocês têm 2 horas para resolver esta prova.
- A interpretação das questões faz parte da avaliação.

**QUESTÃO 01**

Uma startup de e-commerce possui uma aplicação monolítica em Node.js que roda diretamente em um servidor físico. A aplicação tem três componentes principais: um servidor web, um módulo de processamento de pagamentos e um gerador de relatórios. O time decidiu migrar para containers para facilitar o deploy e escalar cada parte independentemente.

**Crie um Dockerfile para um dos três componentes da aplicação** (à sua escolha). O container deve ser funcional, usar uma imagem base. **Desenhe um diagrama mostrando como os três containers se comunicam** entre si e com o mundo externo, incluindo volumes, redes e portas expostas.

- Qual é a diferença entre uma imagem e um container? Por que isso importa no contexto desse deploy?
- Se o módulo de pagamentos exigir variáveis de ambiente sensíveis (chaves de API), como você as gerenciaria sem expor no Dockerfile?
- O que acontece com os dados do banco de dados se o container for reiniciado? Como resolver isso?

**QUESTÃO 02**

Uma equipe de dados mantém dois containers rodando lado a lado: um banco de dados PostgreSQL e uma aplicação de análise em Python. O problema é que toda vez que o container do banco é recriado, os dados somem. Além disso, os dois containers estão na rede padrão do Docker (bridge default) e a aplicação Python precisa se referenciar ao banco pelo IP, que muda a cada restart.

**Escreva os comandos Docker para:** criar um volume nomeado para o banco, criar uma rede personalizada, subir os dois containers conectados a essa rede e montar o volume no container do PostgreSQL.

**Desenhe o diagrama dessa infraestrutura local:** mostre os dois containers, a rede que os conecta, o volume persistindo os dados e o que fica visível para o host.

- Por que numa rede bridge personalizada os containers conseguem se comunicar pelo nome, mas na rede bridge padrão não?
- Se você precisasse que o container Python acessasse um serviço rodando diretamente no host (fora do Docker), como faria isso de forma portátil entre sistemas operacionais?

### QUESTÃO 03

Uma plataforma de vídeo sob demanda precisa suportar picos de acesso durante eventos ao vivo (até 500 mil usuários simultâneos) e manter custos baixos nos períodos normais (cerca de 5 mil usuários). O sistema precisa armazenar vídeos, transcodificá-los em múltiplas resoluções, servir conteúdo com baixa latência globalmente e autenticar usuários.

**Desenhe a arquitetura completa na nuvem** (pode usar AWS, GCP ou Azure. Escolha uma). Identifique os serviços utilizados para cada responsabilidade: armazenamento, processamento, CDN, autenticação e banco de dados. Monte uma tabela de decisão justificando a escolha de pelo menos 3 serviços gerenciados em vez de soluções auto-hospedadas.

- Como a arquitetura garante elasticidade nos picos? Cite os mecanismos específicos.
- Qual é a diferença entre os modelos IaaS, PaaS e SaaS? Identifique onde cada modelo aparece na sua arquitetura.
- Quais são os principais riscos de depender de um único provedor de nuvem? Como mitigá-los?

### QUESTÃO 04

Uma empresa de saúde está avaliando mover seu sistema de prontuários eletrônicos para a nuvem. O CTO tem dúvidas sobre quem é responsável pelo quê: "Se hospedarmos na nuvem, o provedor cuida da segurança ou somos nós?" O sistema lida com dados sensíveis de pacientes, precisa estar disponível 24h e está sujeito a regulamentações de privacidade (como a LGPD). O time de infraestrutura nunca trabalhou com nuvem antes.

**Escreva um parecer técnico curto para o CTO explicando qual modelo de serviço** você recomenda para esse sistema de prontuários e por quê, considerando segurança, conformidade e a curva de aprendizado do time.

- O provedor de nuvem ser responsável pela segurança física da infraestrutura significa que os dados do cliente estão automaticamente protegidos? Explique onde a responsabilidade do cliente começa.
- O que é alta disponibilidade em nuvem e quais conceitos fundamentais (sem citar serviços específicos de nenhum provedor) permitem alcançá-la?
- Nuvem pública, privada e híbrida — qual faz mais sentido para esse cenário de saúde? Justifique considerando custo, controle e conformidade regulatória.

## QUESTÃO 05

Um marketplace online possui um monolito Ruby on Rails com os módulos: cadastro de usuários, catálogo de produtos, carrinho de compras, processamento de pedidos, sistema de pagamentos, notificações (e-mail/SMS) e relatórios financeiros. O time cresceu para 8 squads e o deploy do monolito está causando conflitos e lentidão no ciclo de entrega. O time decidiu migrar para uma arquitetura de microsserviços usando Docker + computação em nuvem (AWS, GCP ou Azure).

Proponha a decomposição em microsserviços considerando:

- Cada serviço deve ser empacotado como uma **imagem Docker** independente, publicada em um container registry gerenciado (ex: Amazon ECR, Google Container Registry ou Azure Container Registry).
- Os containers são executados em um serviço de computação em nuvem sem gerenciamento de servidores (ex: **AWS ECS/Fargate**, **Google Cloud Run** ou **Azure Container Instances**), com escalonamento automático.
- O deploy de cada serviço ocorre via pipeline CI/CD com **docker build + docker push** + atualização da task definition / revisão do serviço na nuvem.

Defina: os serviços, seus limites de responsabilidade e como eles se comunicam. Identifique onde usar **comunicação síncrona** (REST/gRPC entre containers) e **assíncrona** (filas gerenciadas como SQS, Cloud Pub/Sub ou Service Bus).

**a)** O que é o problema do *"distributed monolith"* e como o uso de imagens Docker independentes com deploys desacoplados ajuda a evitar esse anti-pattern? Quais práticas de design de serviço são necessárias para que a independência de deploy seja real?

**b)** Se o container do serviço de Notificações ficar fora do ar (ex: a task do ECS terminar com erro ou o Cloud Run rejeitar requisições por cold start timeout), o fluxo de compra deve ser interrompido? Descreva como projetar resiliência nesse cenário usando filas gerenciadas na nuvem, políticas de retry e dead-letter queues (DLQ). O que acontece com as mensagens enquanto o container está indisponível?